

SQL Interview Questions for SDE Prep

Ranked by Interview Importance – Top 10 → Top 25 → Top 50

TOP 10 (Most Critical – Ask Yourself These First)

Q1: Find the Second Highest Salary

Code:

```
SELECT MAX(salary) AS second_highest_salary
FROM employees
WHERE salary < (SELECT MAX(salary) FROM employees);
```

Explanation:

The inner query finds the overall highest salary. The outer query then finds the maximum salary that is strictly less than that value – which is, by definition, the second highest.

TL;DR: Find the max, then find the max of everything *below* that max.

Key mappings:

- `MAX()` → aggregate function to get the top value
 - Subquery in `WHERE` → filters out the top value before re-aggregating
 - Works even with duplicate top salaries (unlike `LIMIT/OFFSET` in some cases)
-

Q2: Find the Nth Highest Salary

Code:

```
SELECT DISTINCT salary
FROM employees e1
WHERE N - 1 = (
    SELECT COUNT(DISTINCT salary)
    FROM employees e2
```

```
WHERE e2.salary > e1.salary  
);
```

(Replace N with the desired rank, e.g., 3 for third highest)

Alternative (using window functions – preferred in modern SQL):

```
SELECT salary  
FROM (  
    SELECT salary, DENSE_RANK() OVER (ORDER BY salary DESC) AS rnk  
    FROM employees  
) ranked  
WHERE rnk = N;
```

Explanation:

For each salary, count how many distinct salaries are greater than it. If exactly $N-1$ salaries are greater, this salary is the Nth highest. The window function version is cleaner: `DENSE_RANK()` assigns rank 1 to the highest, 2 to the next distinct value, and so on.

TL;DR: Rank all salaries from highest to lowest, then pick the row at rank N.

Key mappings:

- `DENSE_RANK()` → no gaps in ranking even with ties (unlike `RANK()`)
 - `DISTINCT` → avoids counting duplicate salaries as separate ranks
 - This is the single most repeated SQL interview question across companies
-

Q3: Find Duplicate Records in a Table

Code:

```
SELECT name, email, COUNT(*) AS occurrence_count  
FROM users  
GROUP BY name, email  
HAVING COUNT(*) > 1;
```

Explanation:

Grouping by the columns that define a "duplicate" (here, name and email) collapses identical rows into one group. `HAVING COUNT(*) > 1` then filters to only show groups that appeared more than once – i.e., duplicates.

TL;DR: Group identical rows together, count them, keep only groups with count greater than 1.

Key mappings:

- `GROUP BY` → collapses rows with same values
 - `HAVING` → filters groups (unlike `WHERE`, which filters rows before grouping)
 - `COUNT (*)` → counts rows per group
-

Q4: Delete Duplicate Rows, Keeping Only One Copy

Code:

```
DELETE FROM users
WHERE id NOT IN (
    SELECT MIN(id)
    FROM users
    GROUP BY name, email
);
```

Explanation:

For every group of duplicates (same name + email), we keep the row with the smallest `id` and delete everything else. The inner query finds all the "keeper" IDs; the outer `DELETE` removes anything not in that keeper list.

TL;DR: Keep the earliest (smallest ID) copy of each duplicate group, delete the rest.

Key mappings:

- `MIN(id)` → picks one survivor per duplicate group
 - `NOT IN` → deletes everything except survivors
 - Watch for `NULL` values in `NOT IN` subqueries — they can silently break this pattern
-

Q5: Employees Earning More Than Their Manager

Code:

```
SELECT e.name AS employee_name, e.salary, m.name AS manager_name, m.salary
FROM employees e
JOIN employees m ON e.manager_id = m.id
WHERE e.salary > m.salary;
```

Explanation:

This is a **self-join** – the `employees` table is joined to itself, once representing the employee (`e`) and once representing their manager (`m`), matched via `manager_id = id` . The `WHERE` clause then filters for cases where the employee's salary exceeds the manager's.

TL;DR: Join the table to itself so each employee row sits next to their manager's row, then compare salaries.

Key mappings:

- Self-join → same table, two aliases (`e` and `m`)
 - `manager_id = id` → the relationship that links employee to manager
 - Classic test of whether you understand self-joins
-

Q6: Department-Wise Highest Salary

Code:

```
SELECT d.department_name, e.name, e.salary
FROM employees e
JOIN departments d ON e.department_id = d.id
WHERE e.salary = (
    SELECT MAX(salary)
    FROM employees e2
    WHERE e2.department_id = e.department_id
);
```

Alternative (window function):

```
SELECT department_name, name, salary
FROM (
    SELECT d.department_name, e.name, e.salary,
           RANK() OVER (PARTITION BY e.department_id ORDER BY e.salary DE
    FROM employees e
    JOIN departments d ON e.department_id = d.id
) t
WHERE rnk = 1;
```

Explanation:

The correlated subquery version compares each employee's salary to the max salary *within their own*

department. The window function version partitions employees by department and ranks salaries within each partition — much more efficient for large datasets.

TL;DR: For each department, find who earns the most — either by comparing to a per-department max, or by ranking within department groups.

Key mappings:

- `PARTITION BY` → resets ranking per group (here, per department)
 - Correlated subquery → inner query references the outer query's row
 - Tests both join skills and aggregate/window function fluency
-

Q7: Find Employees Without a Department (Join Types)

Code:

```
SELECT e.name
FROM employees e
LEFT JOIN departments d ON e.department_id = d.id
WHERE d.id IS NULL;
```

Explanation:

A `LEFT JOIN` keeps all rows from `employees` regardless of whether a match exists in `departments`. When no match exists, all columns from `departments` come back as `NULL`. Filtering for `d.id IS NULL` isolates employees with no matching department.

TL;DR: Keep all employees, attach department info where it exists, then keep only the ones where nothing attached.

Key mappings:

- `LEFT JOIN` → all rows from left table, matched rows from right (or NULL)
 - `IS NULL` on the joined table's key → the standard "no match" detection pattern
 - Interviewers use this to test if you truly understand JOIN types, not just memorize syntax
-

Q8: Count Employees per Department (Including Departments with Zero Employees)

Code:

```
SELECT d.department_name, COUNT(e.id) AS employee_count
FROM departments d
LEFT JOIN employees e ON d.id = e.department_id
GROUP BY d.department_name;
```

Explanation:

Starting the `LEFT JOIN` from `departments` (not `employees`) ensures every department appears, even if no employee is linked. `COUNT(e.id)` counts only non-NULL employee IDs, so empty departments correctly show `0`.

TL;DR: Start from the table you want *all rows* of, left join the other table, then count carefully (count a column, not `*`).

Key mappings:

- `COUNT(e.id)` vs `COUNT(*)` → `COUNT(*)` would wrongly count 1 for empty departments; `COUNT(column)` ignores NULLs
 - Direction of `LEFT JOIN` matters – this trips up many candidates
-

Q9: Running Total / Cumulative Sum

Code:

```
SELECT order_date, amount,
       SUM(amount) OVER (ORDER BY order_date) AS running_total
FROM orders;
```

Explanation:

The window function `SUM(amount) OVER (ORDER BY order_date)` computes a cumulative sum – for each row, it adds up `amount` from the first row up through the current row, ordered by date.

TL;DR: Use `SUM() OVER (ORDER BY ...)` to get a "total so far" column without collapsing rows via `GROUP BY`.

Key mappings:

- `OVER (ORDER BY ...)` → defines the running window
 - Window functions keep every row (unlike `GROUP BY`, which collapses rows)
 - Foundational for time-series and financial SQL questions
-

Q10: RANK() vs DENSE_RANK() vs ROW_NUMBER()

Code:

```
SELECT name, salary,  
       RANK() OVER (ORDER BY salary DESC) AS rank_val,  
       DENSE_RANK() OVER (ORDER BY salary DESC) AS dense_rank_val,  
       ROW_NUMBER() OVER (ORDER BY salary DESC) AS row_num  
FROM employees;
```

Explanation:

All three assign a position to each row based on `salary DESC`, but they handle ties differently:

- `ROW_NUMBER()` → always unique, sequential (1,2,3,4...), even for ties
- `RANK()` → ties get the same rank, but the next rank skips numbers (1,2,2,4)
- `DENSE_RANK()` → ties get the same rank, no gap in the next rank (1,2,2,3)

TL;DR: ROW_NUMBER never repeats. RANK repeats and then skips. DENSE_RANK repeats and never skips.

Key mappings:

- These three are asked constantly to test if you understand tie-handling
- Almost every "top N per group" or "Nth highest" question builds on this concept

TOP 11–25 (Very Commonly Asked)

Q11: Find Duplicate Emails in a Table

Code:

```
SELECT email  
FROM users  
GROUP BY email  
HAVING COUNT(*) > 1;
```

Explanation:

Groups rows by email, then filters to only groups that appear more than once.

TL;DR: Same duplicate-detection pattern as Q3, but simplified to a single column.

Key mappings:

- `GROUP BY + HAVING COUNT(*) > 1` → the universal "find duplicates" template
-

Q12: Consecutive Numbers Appearing at Least 3 Times

Code:

```
SELECT DISTINCT l1.num AS consecutive_nums
FROM logs l1
JOIN logs l2 ON l1.id = l2.id - 1
JOIN logs l3 ON l1.id = l3.id - 2
WHERE l1.num = l2.num AND l2.num = l3.num;
```

Explanation:

Three aliases of the same table are joined on consecutive `id` values ($i, i+1, i+2$). If the `num` value is identical across all three, that number occurs at least three times in a row.

TL;DR: Chain-join a table to itself across consecutive IDs, then check if the value repeats across the chain.

Key mappings:

- Multi-way self-join → common LeetCode-style pattern
 - Tests understanding of sequential/positional logic in SQL
-

Q13: Employees with the Same Salary

Code:

```
SELECT e1.name AS employee1, e2.name AS employee2, e1.salary
FROM employees e1
JOIN employees e2 ON e1.salary = e2.salary AND e1.id < e2.id;
```

Explanation:

Self-join on matching salary, with `e1.id < e2.id` ensuring each pair is only shown once (avoids showing both A-B and B-A, and avoids matching an employee with themselves).

TL;DR: Join the table to itself on salary, use an ID inequality to avoid duplicate/mirror pairs.

Key mappings:

- `e1.id < e2.id` → the standard trick to prevent duplicate pair-matching in self-joins
-

Q14: Top N Salaries per Department

Code:

```
SELECT department_name, name, salary
FROM (
    SELECT d.department_name, e.name, e.salary,
           DENSE_RANK() OVER (PARTITION BY e.department_id ORDER BY e.sal
    FROM employees e
    JOIN departments d ON e.department_id = d.id
) ranked
WHERE rnk <= N;
```

Explanation:

Partitioning by department and ranking salary within each partition gives a per-department leaderboard. Filtering `rnk <= N` keeps the top N earners in every department.

TL;DR: Rank employees within each department by salary, then keep only the top N ranks.

Key mappings:

- `PARTITION BY` → resets rank counting for each department
 - Generalized version of Q6
-

Q15: Calculate the Median Salary

Code:

```
SELECT AVG(salary) AS median_salary
FROM (
    SELECT salary,
           ROW_NUMBER() OVER (ORDER BY salary) AS row_asc,
           ROW_NUMBER() OVER (ORDER BY salary DESC) AS row_desc
    FROM employees
) t
WHERE row_asc = row_desc
```

```
OR row_asc + 1 = row_desc
OR row_desc + 1 = row_asc;
```

Explanation:

Ranking rows both ascending and descending finds the middle row(s). If total rows is odd, one row satisfies `row_asc = row_desc`. If even, two middle rows satisfy the "off by one" conditions, and averaging them gives the median.

TL;DR: Find the row(s) sitting exactly in the middle when sorted, then average if there are two.

Key mappings:

- Dual `ROW_NUMBER()` (ascending and descending) → the standard median trick in plain SQL
 - Some databases have native `MEDIAN()` or `PERCENTILE_CONT()` functions – mention these as alternatives
-

Q16: Pivot Rows into Columns (Conditional Aggregation)

Code:

```
SELECT
    department_id,
    SUM(CASE WHEN gender = 'M' THEN 1 ELSE 0 END) AS male_count,
    SUM(CASE WHEN gender = 'F' THEN 1 ELSE 0 END) AS female_count
FROM employees
GROUP BY department_id;
```

Explanation:

`CASE WHEN` inside `SUM()` converts each row into a 1 or 0 depending on a condition, then aggregates. This effectively "pivots" category values (M/F) into their own columns.

TL;DR: Use CASE inside SUM/COUNT to turn row values into separate summary columns.

Key mappings:

- `CASE WHEN ... THEN 1 ELSE 0 END` → the manual pivot pattern, works in all SQL dialects
 - Some databases (SQL Server, Oracle) have native `PIVOT` syntax as an alternative
-

Q17: Date Difference Between Two Events

Code:

```
SELECT order_id, order_date, delivery_date,  
       DATEDIFF(delivery_date, order_date) AS days_to_deliver  
FROM orders;
```

(In PostgreSQL: *delivery_date - order_date*)

Explanation:

`DATEDIFF()` (MySQL/SQL Server syntax) calculates the number of days between two dates. PostgreSQL allows direct subtraction of date types to get an interval/integer.

TL;DR: Subtract two dates (or use `DATEDIFF`) to get the number of days between them.

Key mappings:

- Date function syntax varies by database – always clarify which SQL dialect the interview expects
 - Common follow-up: "find average delivery time" using `AVG(DATEDIFF(...))`
-

Q18: String Aggregation (Combine Multiple Rows into One String)

Code:

```
SELECT department_id, GROUP_CONCAT(name SEPARATOR ', ') AS employee_names  
FROM employees  
GROUP BY department_id;
```

(In PostgreSQL: *STRING_AGG(name, ', ')*)

Explanation:

`GROUP_CONCAT` (MySQL) or `STRING_AGG` (PostgreSQL/SQL Server) combines multiple row values within a group into a single comma-separated string.

TL;DR: Merge multiple text rows in a group into one combined string, separated by a delimiter.

Key mappings:

- `GROUP_CONCAT` → MySQL syntax
 - `STRING_AGG` → PostgreSQL/SQL Server syntax
 - Mention dialect differences to show real-world SQL awareness
-

Q19: Find Missing IDs in a Sequence

Code:

```
SELECT (t1.id + 1) AS missing_id
FROM orders t1
WHERE NOT EXISTS (
    SELECT 1 FROM orders t2 WHERE t2.id = t1.id + 1
)
AND t1.id + 1 < (SELECT MAX(id) FROM orders);
```

Explanation:

For every ID, check whether `id + 1` exists in the table. If it doesn't (and it's below the max ID), then `id + 1` is a gap in the sequence.

TL;DR: Check each ID's "next number" — if that next number isn't in the table, it's missing.

Key mappings:

- `NOT EXISTS` → efficient way to check absence of a matching row
 - Common in "find gaps" style questions (IDs, dates, invoice numbers)
-

Q20: Customers Who Never Placed an Order

Code:

```
SELECT c.name
FROM customers c
LEFT JOIN orders o ON c.id = o.customer_id
WHERE o.id IS NULL;
```

Alternative:

```
SELECT name
FROM customers
WHERE id NOT IN (SELECT customer_id FROM orders WHERE customer_id IS NOT
```

Explanation:

Same "anti-join" pattern as Q7 — `LEFT JOIN` plus `IS NULL` finds customers with no matching order row. The `NOT IN` alternative works too, but requires excluding NULLs from the subquery explicitly to avoid silently returning zero rows.

TL;DR: Left join customers to orders, keep only customers where no order matched.

Key mappings:

- Anti-join pattern → reused constantly across interview questions
 - `NOT IN` + NULLs → a classic SQL gotcha worth mentioning to impress the interviewer
-

Q21: Percentage of Total (Window Function)

Code:

```
SELECT product_name, sales,  
       ROUND(100.0 * sales / SUM(sales) OVER (), 2) AS pct_of_total  
FROM product_sales;
```

Explanation:

`SUM(sales) OVER ()` (empty `OVER()`) computes the grand total across all rows without collapsing them. Dividing each row's sales by that total gives the percentage contribution per row.

TL;DR: Use an empty `OVER()` to get a grand total alongside every row, then divide.

Key mappings:

- `OVER ()` with no `PARTITION BY / ORDER BY` → total across the entire result set
 - Tests deeper window function understanding beyond ranking
-

Q22: LEAD() and LAG() – Compare Row to Previous/Next Row

Code:

```
SELECT employee_id, month, salary,  
       LAG(salary) OVER (PARTITION BY employee_id ORDER BY month) AS prev  
       LEAD(salary) OVER (PARTITION BY employee_id ORDER BY month) AS nex  
FROM salary_history;
```

Explanation:

`LAG()` pulls a value from the previous row in the ordered partition; `LEAD()` pulls from the next row. This is extremely useful for month-over-month or day-over-day comparisons without a self-join.

TL;DR: LAG looks backward one row, LEAD looks forward one row – both within a defined order.

Key mappings:

- `LAG / LEAD` → replace many self-join use cases with cleaner window functions
 - Common follow-up: calculate month-over-month salary change using `salary - LAG(salary)`
-

Q23: Update Table Using a Join

Code:

```
UPDATE employees e
JOIN departments d ON e.department_id = d.id
SET e.department_name = d.department_name
WHERE d.is_active = 1;
```

(PostgreSQL syntax differs – uses `UPDATE ... FROM`)

Explanation:

An `UPDATE` combined with a `JOIN` lets you set column values on one table based on matching data in another. Only rows satisfying the `WHERE` and `JOIN ON` conditions get updated.

TL;DR: Join two tables inside an `UPDATE` statement so you can copy/sync values from one table into another.

Key mappings:

- Syntax differs significantly across MySQL/PostgreSQL/SQL Server – worth noting in the interview
 - Tests real-world data maintenance skills, not just `SELECT` queries
-

Q24: Delete Duplicate Rows (Keeping Latest Instead of Earliest)

Code:

```
DELETE FROM users
WHERE id NOT IN (
    SELECT MAX(id)
    FROM users
    GROUP BY name, email
);
```

Explanation:

Variation of Q4 – using `MAX(id)` instead of `MIN(id)` keeps the most recently inserted row

(assuming auto-incrementing IDs) rather than the earliest.

TL;DR: Same duplicate-deletion logic as before, but keep the newest row instead of the oldest.

Key mappings:

- `MAX(id)` vs `MIN(id)` → shows you understand why the choice matters, not just the syntax
-

Q25: Recursive CTE – Employee Hierarchy / Manager Chain

Code:

```
WITH RECURSIVE employee_hierarchy AS (  
    SELECT id, name, manager_id, 1 AS level  
    FROM employees  
    WHERE manager_id IS NULL  
  
    UNION ALL  
  
    SELECT e.id, e.name, e.manager_id, eh.level + 1  
    FROM employees e  
    JOIN employee_hierarchy eh ON e.manager_id = eh.id  
)  
SELECT * FROM employee_hierarchy;
```

Explanation:

The "anchor" query selects the top-level employees (those with no manager). The recursive part repeatedly joins the next level of employees to the previous level, incrementing `level` each time, until no more matches are found.

TL;DR: Start at the top of a hierarchy, then repeatedly join "one level down" until you reach the bottom.

Key mappings:

- `WITH RECURSIVE` → defines a self-referencing CTE
 - Anchor query (base case) + recursive query (repeats) → the two required parts
 - A strong signal question – few candidates handle this smoothly
-

 **TOP 26–50 (Good to Know – Rounds Out Strong Preparation)**

Q26: First and Last Record per Group

Code:

```
SELECT department_id, name, hire_date
FROM (
    SELECT *,
           ROW_NUMBER() OVER (PARTITION BY department_id ORDER BY hire_date) first_rn,
           ROW_NUMBER() OVER (PARTITION BY department_id ORDER BY hire_date DESC) last_rn
    FROM employees
) t
WHERE first_rn = 1 OR last_rn = 1;
```

Explanation: Two separate `ROW_NUMBER()` rankings (ascending and descending by hire date) let you isolate the earliest and latest hire in each department.

TL;DR: Rank each group both ways (earliest-first and latest-first), then grab rank 1 from each.

Key mappings: `PARTITION BY` with dual ordering → reusable pattern for "first/last per group" questions.

Q27: Moving Average (Last 3 Rows)

Code:

```
SELECT order_date, amount,
       AVG(amount) OVER (ORDER BY order_date ROWS BETWEEN 2 PRECEDING AND CURRENT ROW) moving_avg
FROM orders;
```

Explanation: The `ROWS BETWEEN 2 PRECEDING AND CURRENT ROW` frame tells the window function to only look at the current row plus the two before it, averaging just that sliding window.

TL;DR: Use a windowed frame (`ROWS BETWEEN`) to average only the last N rows instead of everything up to now.

Key mappings: Frame clause (`ROWS BETWEEN ... AND ...`) → controls exactly which rows the window function considers.

Q28: Find Gaps in a Sequence of Dates

Code:

```
SELECT log_date,  
       LEAD(log_date) OVER (ORDER BY log_date) AS next_date,  
       DATEDIFF(LEAD(log_date) OVER (ORDER BY log_date), log_date) AS gap_days  
FROM activity_log  
QUALIFY gap_days > 1;
```

(If *QUALIFY* isn't supported, wrap in a subquery and filter with *WHERE* instead)

Explanation: *LEAD()* grabs the next date in sequence; comparing it to the current date reveals any gap larger than one day.

TL;DR: Compare each date to the next date using *LEAD*, flag anywhere the gap is more than 1 day.

Key mappings: *LEAD()* → same tool as Q22, applied to date-gap detection instead of value comparison.

Q29: CASE Statement for Conditional Labeling

Code:

```
SELECT name, salary,  
       CASE  
           WHEN salary >= 100000 THEN 'High'  
           WHEN salary >= 50000 THEN 'Medium'  
           ELSE 'Low'  
       END AS salary_band  
FROM employees;
```

Explanation: *CASE WHEN* evaluates conditions top to bottom and returns the label for the first matching condition, similar to if-else logic in programming.

TL;DR: *CASE WHEN* is SQL's if-else — checks conditions in order and returns a label for the first match.

Key mappings: *CASE WHEN ... ELSE ... END* → foundational, reused in Q16 and Q37.

Q30: Joining Three Tables Together

Code:

```
SELECT o.order_id, c.name AS customer_name, p.product_name
FROM orders o
JOIN customers c ON o.customer_id = c.id
JOIN products p ON o.product_id = p.id;
```

Explanation: Each **JOIN** links one more table into the result. Order of joins generally doesn't affect correctness (just readability/performance), as long as the **ON** conditions correctly connect each pair.

TL;DR: Chain JOINS one after another — each new JOIN links in one more table.

Key mappings: Multi-table joins → tests whether you can extend two-table logic to real-world schemas with 3+ tables.

Q31: Subquery vs JOIN (Conceptual + Code)

Code:

```
-- Using JOIN
SELECT DISTINCT c.name
FROM customers c
JOIN orders o ON c.id = o.customer_id;

-- Using Subquery
SELECT name
FROM customers
WHERE id IN (SELECT customer_id FROM orders);
```

Explanation: Both return customers who have placed orders. JOINS are generally more flexible (can pull columns from both tables) and often perform better with proper indexing; subqueries can be more readable for simple existence checks.

TL;DR: JOIN and subquery can solve the same problem — JOIN when you need columns from both tables, subquery when you just need a filter.

Key mappings: Conceptual question — expect a follow-up on performance trade-offs and **EXPLAIN** plans.

Q32: Correlated Subquery Example

Code:

```
SELECT name, salary, department_id
FROM employees e1
WHERE salary > (
    SELECT AVG(salary)
    FROM employees e2
    WHERE e2.department_id = e1.department_id
);
```

Explanation: The inner query re-runs for every row of the outer query, using `e1.department_id` from the current outer row. This finds employees earning more than their own department's average.

TL;DR: A correlated subquery re-executes once per outer row, referencing that row's values — unlike a normal subquery which runs once.

Key mappings: "Correlated" = inner query depends on outer query's current row → key distinction interviewers test for.

Q33: UNION vs UNION ALL

Code:

```
-- Removes duplicates
SELECT city FROM customers
UNION
SELECT city FROM suppliers;

-- Keeps duplicates
SELECT city FROM customers
UNION ALL
SELECT city FROM suppliers;
```

Explanation: `UNION` combines two result sets and removes duplicate rows (which requires an internal sort/dedup step). `UNION ALL` combines them without removing duplicates, making it faster.

TL;DR: `UNION` = combine and deduplicate (slower). `UNION ALL` = combine and keep everything (faster).

Key mappings: Performance nuance (`UNION ALL` avoids the dedup cost) is a favorite follow-up question.

Q34: Employees Hired in the Last N Days

Code:

```
SELECT name, hire_date
FROM employees
WHERE hire_date >= CURDATE() - INTERVAL 30 DAY;
```

(PostgreSQL: *CURRENT_DATE - INTERVAL '30 days'*)

Explanation: Subtracting an interval from the current date gives a cutoff date; filtering `hire_date` against that cutoff returns recent hires.

TL;DR: Compare a date column to "today minus N days" to filter recent records.

Key mappings: Date arithmetic syntax varies by dialect — always confirm which database the interviewer means.

Q35: Cumulative Distinct Count

Code:

```
SELECT order_date,
       COUNT(DISTINCT customer_id) OVER (ORDER BY order_date) AS cumulative_count
FROM orders;
```

Explanation: Note: not all databases support `DISTINCT` inside a window function directly (e.g., standard MySQL doesn't). Where unsupported, this typically requires a self-join or correlated subquery workaround instead.

TL;DR: Ideally a windowed distinct count — but be ready to explain the workaround since many databases don't support it natively.

Key mappings: A great question to show awareness of database-specific limitations, not just textbook syntax.

Q36: Salary Bucket / Histogram Using CASE

Code:

```
SELECT
  CASE
    WHEN salary < 30000 THEN '0-30k'
```

```
        WHEN salary < 60000 THEN '30k-60k'
        WHEN salary < 100000 THEN '60k-100k'
        ELSE '100k+'
    END AS salary_bucket,
    COUNT(*) AS employee_count
FROM employees
GROUP BY salary_bucket;
```

Explanation: `CASE` buckets each salary into a range label, then `GROUP BY` on that computed label produces a histogram-style count per bucket.

TL;DR: Label each row into a bucket with `CASE`, then group and count by that bucket label.

Key mappings: Combines Q16 and Q29's patterns into one applied use case — grouping by a derived (non-column) value.

Q37: Find Overlapping Date Ranges

Code:

```
SELECT b1.booking_id, b2.booking_id AS overlapping_booking_id
FROM bookings b1
JOIN bookings b2 ON b1.room_id = b2.room_id
                  AND b1.booking_id < b2.booking_id
                  AND b1.start_date < b2.end_date
                  AND b2.start_date < b1.end_date;
```

Explanation: Two date ranges overlap if one starts before the other ends, in both directions. Self-joining on the same room and applying that overlap condition surfaces conflicting bookings.

TL;DR: Two ranges overlap when each one starts before the other ends — self-join and check that condition both ways.

Key mappings: Classic scheduling/booking-system interview question, tests boundary-condition thinking.

Q38: Second Highest Salary Without LIMIT/OFFSET

Code:

```
SELECT MAX(salary) AS second_highest
FROM employees
WHERE salary <> (SELECT MAX(salary) FROM employees);
```

Explanation: Same core idea as Q1, but phrased to explicitly avoid `LIMIT / OFFSET` (which some interviewers disallow to test pure logic, since not all databases support `LIMIT` the same way — e.g., SQL Server uses `TOP / OFFSET FETCH`).

TL;DR: Exclude the overall max, then take the max of what's left — no `LIMIT` needed at all.

Key mappings: Shows portability awareness — this approach works identically across MySQL, PostgreSQL, SQL Server, and Oracle.

Q39: Employees Who Joined Before Their Manager

Code:

```
SELECT e.name AS employee_name, e.hire_date, m.name AS manager_name, m.hi
FROM employees e
JOIN employees m ON e.manager_id = m.id
WHERE e.hire_date < m.hire_date;
```

Explanation: Another self-join (like Q5), but comparing hire dates instead of salaries — finds employees who were hired earlier than the manager they now report to.

TL;DR: Same self-join structure as the manager-salary question, just comparing hire dates instead.

Key mappings: Shows you can generalize the self-join pattern to any comparable attribute, not just salary.

Q40: Convert Row Values into Columns (Manual Pivot with Labels)

Code:

```
SELECT
    year,
    SUM(CASE WHEN quarter = 'Q1' THEN revenue ELSE 0 END) AS Q1_revenue,
    SUM(CASE WHEN quarter = 'Q2' THEN revenue ELSE 0 END) AS Q2_revenue,
    SUM(CASE WHEN quarter = 'Q3' THEN revenue ELSE 0 END) AS Q3_revenue,
    SUM(CASE WHEN quarter = 'Q4' THEN revenue ELSE 0 END) AS Q4_revenue
```

```
FROM quarterly_sales
GROUP BY year;
```

Explanation: Extension of Q16 — each quarter's revenue becomes its own summed column, giving a wide-format report from long-format data.

TL;DR: One SUM(CASE...) column per category value — turns tall data into a wide report.

Key mappings: Real-world reporting pattern — commonly asked in data-analyst-leaning SDE interviews.

Q41: Calculate Age from Date of Birth

Code:

```
SELECT name, dob,
       TIMESTAMPDIFF(YEAR, dob, CURDATE()) AS age
FROM users;
```

(PostgreSQL: `EXTRACT(YEAR FROM AGE(CURRENT_DATE, dob))`)

Explanation: `TIMESTAMPDIFF` (or equivalent) computes the number of full years between the birth date and today, correctly handling whether the birthday has occurred yet this year.

TL;DR: Use the database's date-diff function in "years" mode to get an accurate current age.

Key mappings: Watch for the classic bug: simple year subtraction (`YEAR(CURDATE()) - YEAR(dob)`) is wrong if the birthday hasn't happened yet this year.

Q42: Customers Who Ordered in Every Month of a Year

Code:

```
SELECT customer_id
FROM orders
WHERE YEAR(order_date) = 2025
GROUP BY customer_id
HAVING COUNT(DISTINCT MONTH(order_date)) = 12;
```

Explanation: Grouping by customer and counting distinct order months tells you how many unique months they ordered in. Requiring that count to equal 12 means they ordered every single month.

TL;DR: Count distinct months ordered per customer – keep only customers who hit all 12.

Key mappings: `COUNT (DISTINCT ...)` inside `HAVING` → combines multiple concepts (grouping, distinct counting, filtering groups) in one question.

Q43: Nth Occurrence of a Repeated Value

Code:

```
SELECT *
FROM (
    SELECT *, ROW_NUMBER() OVER (PARTITION BY user_id ORDER BY login_time)
    FROM logins
) t
WHERE occurrence_num = N;
```

Explanation: Partitioning by the repeated value (e.g., `user_id`) and numbering rows within each partition lets you isolate exactly the Nth time that value appeared.

TL;DR: Number each occurrence of a value within its own group, then filter to the Nth number.

Key mappings: `PARTITION BY` + `ROW_NUMBER()` → reused pattern, applied here to "occurrence counting" rather than ranking.

Q44: Query Optimization Awareness (Conceptual)

Code:

```
-- Less optimal: function on indexed column blocks index usage
SELECT * FROM orders WHERE YEAR(order_date) = 2025;

-- More optimal: keeps index usable
SELECT * FROM orders
WHERE order_date >= '2025-01-01' AND order_date < '2026-01-01';
```

Explanation: Wrapping an indexed column in a function (like `YEAR()`) usually forces the database to evaluate that function for every row, preventing it from using an index efficiently. Rewriting the condition as a direct range comparison lets the database use the index (via an index seek) instead.

TL;DR: Avoid wrapping indexed columns in functions in WHERE clauses – rewrite as direct range comparisons so indexes still work.

Key mappings: "Sargable" queries → term for conditions that can use an index; strong candidates bring this up unprompted.

Q45: CTE vs Subquery (When to Use Which)

Code:

```
-- Using CTE
WITH high_earners AS (
    SELECT * FROM employees WHERE salary > 100000
)
SELECT department_id, COUNT(*) FROM high_earners GROUP BY department_id;

-- Using Subquery
SELECT department_id, COUNT(*)
FROM (SELECT * FROM employees WHERE salary > 100000) AS high_earners
GROUP BY department_id;
```

Explanation: Both produce identical results, but a CTE (`WITH ... AS`) is named and can be reused multiple times in the same query, and reads top-down like a sequence of logical steps – making complex queries easier to follow and maintain.

TL;DR: CTEs and subqueries can do the same job – CTEs win on readability and reusability within one query.

Key mappings: `WITH` clause → foundational for Q25's recursive CTE; shows query-structuring maturity.

Q46: Second Highest Salary per Department (Combining Concepts)

Code:

```
SELECT department_id, name, salary
FROM (
    SELECT department_id, name, salary,
           DENSE_RANK() OVER (PARTITION BY department_id ORDER BY salary)
    FROM employees
```

```
) ranked  
WHERE rnk = 2;
```

Explanation: Combines Q2's ranking logic with Q6/Q14's partitioning-by-department logic to get the second-highest earner in every department at once.

TL;DR: Rank salaries within each department, then filter to rank 2 everywhere.

Key mappings: A favorite "combine two things you've already learned" interview question — shows if you can compose patterns, not just memorize them.

Q47: Find Rows That Exist in One Table but Not Another

Code:

```
SELECT product_id FROM warehouse_a  
EXCEPT  
SELECT product_id FROM warehouse_b;
```

(MySQL, which lacks *EXCEPT*, uses: *LEFT JOIN ... WHERE b.id IS NULL*)

Explanation: *EXCEPT* (PostgreSQL/SQL Server) returns rows from the first query that don't appear in the second. It's a set-based way to find "what's missing" between two tables.

TL;DR: *EXCEPT* subtracts one result set from another, similar to a set difference in math.

Key mappings: *EXCEPT* / *MINUS* → dialect-specific; always have the *LEFT JOIN + IS NULL* fallback ready for MySQL.

Q48: ACID Properties (Conceptual, with a Practical Example)

Code:

```
BEGIN TRANSACTION;  
  
UPDATE accounts SET balance = balance - 500 WHERE account_id = 1;  
UPDATE accounts SET balance = balance + 500 WHERE account_id = 2;  
  
COMMIT;
```

Explanation: This is a classic money-transfer example. **Atomicity** ensures both updates succeed or both roll back together. **Consistency** ensures the total balance across accounts stays valid. **Isolation** ensures concurrent transactions don't see partial updates. **Durability** ensures the committed change survives a crash.

TL;DR: ACID = the transaction either fully happens or fully doesn't, stays valid, doesn't interfere with others, and survives once committed.

Key mappings: `BEGIN TRANSACTION` / `COMMIT` / `ROLLBACK` → the practical commands behind the ACID theory question.

Q49: Database Normalization Example (Design Question)

Code:

```
-- Unnormalized (repeating group in one row)
-- orders(order_id, customer_name, product1, product2, product3)

-- Normalized (1NF/2NF/3NF applied)
CREATE TABLE customers (customer_id INT PRIMARY KEY, customer_name VARCHAR(255));

CREATE TABLE orders (order_id INT PRIMARY KEY, customer_id INT,
                     FOREIGN KEY (customer_id) REFERENCES customers(customer_id));

CREATE TABLE order_items (order_item_id INT PRIMARY KEY, order_id INT, product VARCHAR(255),
                          FOREIGN KEY (order_id) REFERENCES orders(order_id));
```

Explanation: The unnormalized design repeats product columns and duplicates customer names across rows, wasting space and risking inconsistency. Splitting into `customers`, `orders`, and `order_items` removes repeating groups (1NF), removes partial dependencies (2NF), and removes transitive dependencies (3NF).

TL;DR: Break one messy table with repeated/duplicated data into clean, linked tables – one concept per table.

Key mappings: `PRIMARY KEY` / `FOREIGN KEY` → the mechanism that enforces the relationships created by normalization.

Q50: Find the Top 1 Row per Group Without Window Functions (Legacy/Older SQL)

Code:

```
SELECT e1.department_id, e1.name, e1.salary
FROM employees e1
LEFT JOIN employees e2
      ON e1.department_id = e2.department_id AND e1.salary < e2.salary
WHERE e2.id IS NULL;
```

Explanation: This self-join checks, for each employee, whether any other employee in the same department earns more. If no such employee exists (`e2.id IS NULL`), the current employee is the top earner in their department. This achieves the same result as Q6's window function version, using only joins — useful for older databases without window function support.

TL;DR: For each employee, check if anyone in their department earns more — if nobody does, they're the top earner.

Key mappings: Anti-join pattern applied to "top-1-per-group" → valuable to know for legacy systems (older MySQL versions, some embedded databases) that lack window functions.

Quick Reference: Most Reused Patterns Across All 50 Questions

Pattern	Used In
<code>GROUP BY + HAVING COUNT(*) > 1</code>	Q3, Q11
Self-join(<code>table AS a JOIN table AS b</code>)	Q5, Q12, Q13, Q37, Q39, Q50
<code>LEFT JOIN + IS NULL</code> (anti-join)	Q7, Q8, Q19, Q20, Q47, Q50
<code>RANK()</code> / <code>DENSE_RANK()</code> / <code>ROW_NUMBER()</code>	Q2, Q10, Q14, Q26, Q43, Q46
<code>PARTITION BY</code>	Q6, Q14, Q22, Q26, Q35, Q43, Q46
<code>CASE WHEN</code> (conditional logic)	Q16, Q29, Q36, Q40, Q44
<code>LEAD()</code> / <code>LAG()</code>	Q22, Q28
<code>WITH (CTE)</code> / <code>WITH RECURSIVE</code>	Q25, Q45
Correlated subquery	Q6, Q32, Q38

Study tip: Master the 9 patterns above deeply — nearly all 50 questions are combinations of just these building blocks.